

05. Sorting

Hyunjong Choi, Ph.D.

Assistant Professor of Computer Science

San Diego State University

Overview

- ❑ Quicksort (Textbook 3rd edition 170 – 190)
- ❑ Heapsort ([Textbook 3rd edition 151 – 169](#))
- ❑ Linear-time sorting (Textbook 3rd edition 191 – 212)

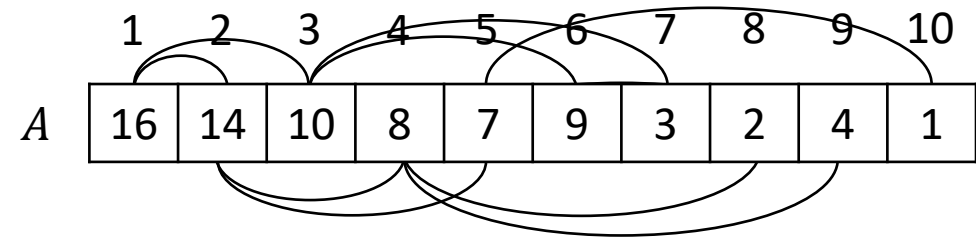
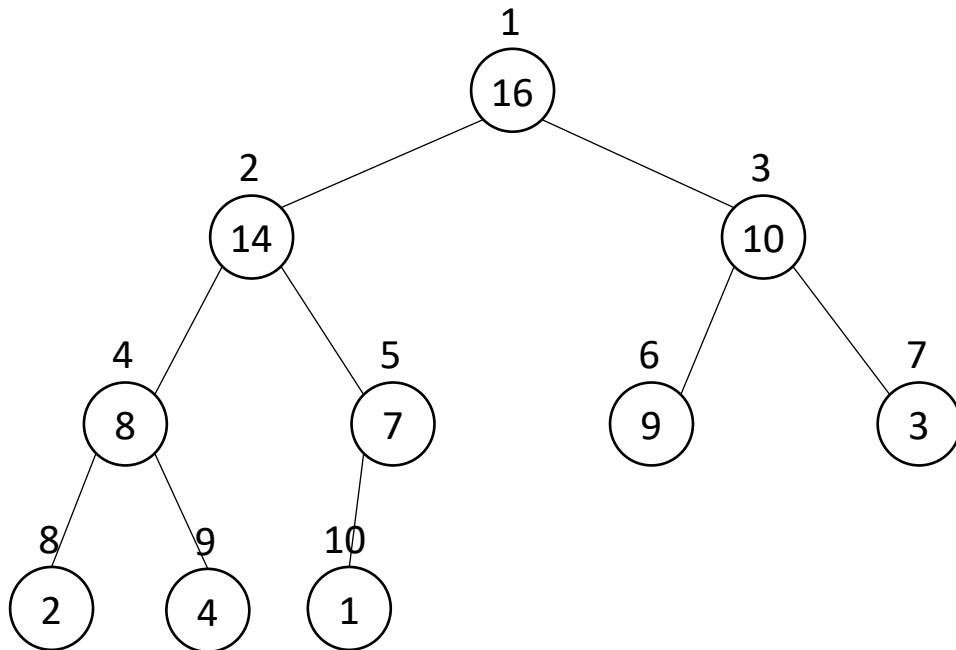


Heapsort

- ❑ Running time is $O(n \lg n)$
 - Like merge sort, quicksort, unlike insertion sort
- ❑ Sorts **in-place**: only a constant number of elements are stored outside the input array
- ❑ Design idea: using a data structure **heap** to manage information

What is heap?

- ❑ A **(binary) heap** data structure is an array object that can be viewed as a nearly complete binary tree



- An array A that represents a heap has two attributes:
 - ✓ $A.length$: the number of elements in the array
 - ✓ $A.heap_size$: the number of elements in the heap

Representation of heap

□ Representation of a heap with an array A :

- Only the elements in $A[1:A.heap_size]$, where $0 \leq A.heap_size \leq A.length$, are valid elements of the heap.
- The root is $A[1]$

□ Parent, left child, and right child with the given index i

PARENT(i)

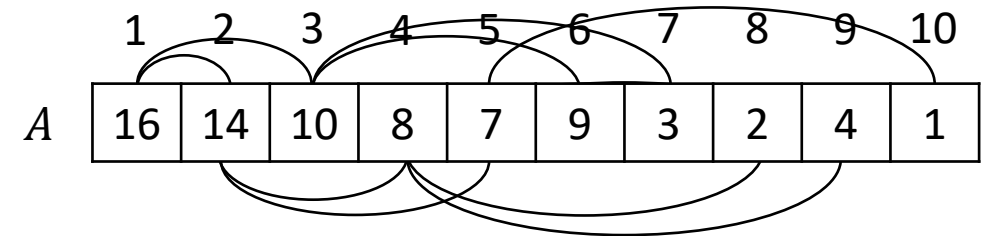
1. **return** $\lfloor i/2 \rfloor$

LEFT(i)

1. **return** $2i$

RIGHT(i)

1. **return** $2i + 1$



Heap property

❑ Two types of binary heaps: ***max-heap*** and ***min-heap***

- A ***max-heap*** satisfies that for every node i other than the root,

$$A[PARENT(i)] \geq A[i]$$

- ✓ Largest element stored at root node
- ✓ Subtrees rooted at any node contains values no larger than the node itself.

- A ***min-heap*** satisfies the opposite, $A[PARENT(i)] \leq A[i]$

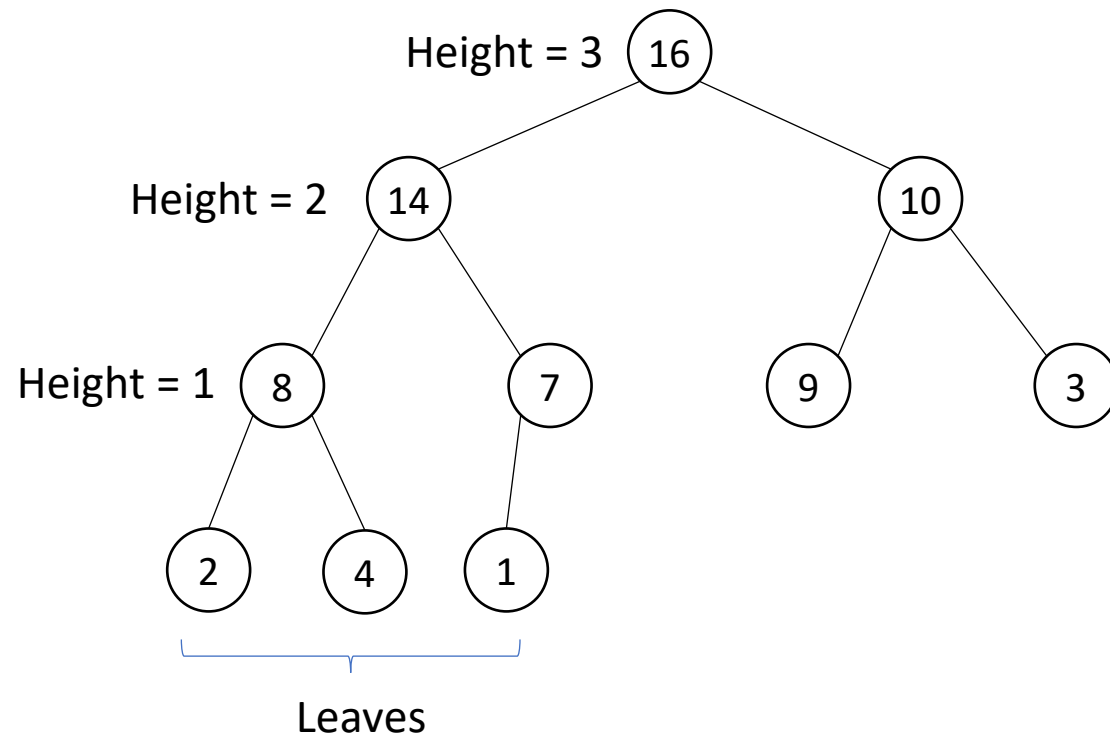
- ✓ Smallest element stored at the root.

❑ Heapsort uses max-heaps.

Height of a heap

□ **Height** of a node defined as:

- The number of edges on the longest downward path from the root to a leaf



- Height of the root node
= Height of the heap
- Height of a heap of n elements is $\Theta(\lg_2 n)$

Basic operations on a heap

MAX- HEAPIFY

- Maintaining the max-heap property
- $O(\lg n)$ time

BUILD-MAX- HEAP

- Creating a max-heap from an unordered array
- $O(n)$ time

HEAPSORT

- Sorting an array in-place
- $O(n \lg n)$ time

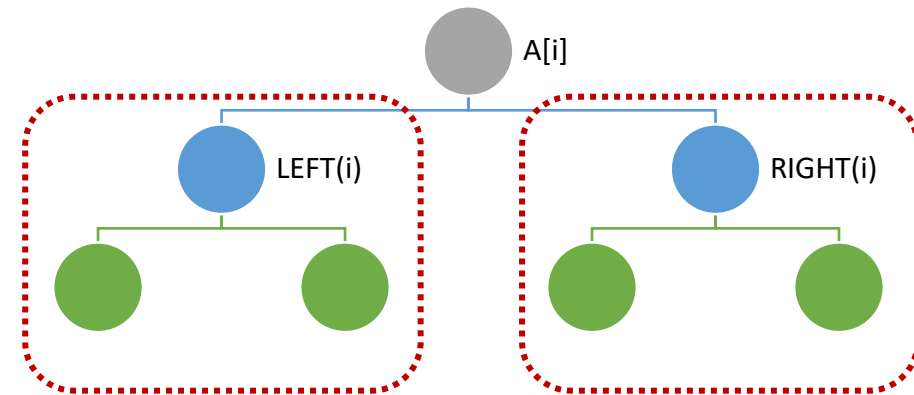
MAX-HEAPIFY procedure

MAX-HEAPIFY

- ❑ Input: array A , index i .
- ❑ Assuming the sub-trees rooted at $\text{LEFT}(i)$ and $\text{RIGHT}(i)$ are already max-heaps, but $A[i]$ might be smaller than its children, thus *violating* the max-heap property.

MAX-HEAPIFY(A, i)

1. $l = \text{LEFT}(i)$
2. $r = \text{RIGHT}(i)$
3. **if** $l \leq A.\text{heap_size}$ and $A[l] > A[i]$
4. $\text{largest} = l$
5. **else** $\text{largest} = i$
6. **if** $r \leq A.\text{heap_size}$ and $A[r] > A[\text{largest}]$
7. $\text{largest} = r$
8. **if** $\text{largest} \neq i$
9. exchange $A[i]$ with $A[\text{largest}]$
10. MAX-HEAPIFY($A, \text{largest}$)



MAX-HEAPIFY lets the value at $A[i]$ “float down” in the max-heap so that the subtree rooted at index i obeys the max-heap property.

MAX-HEAPIFY procedure

MAX-HEAPIFY

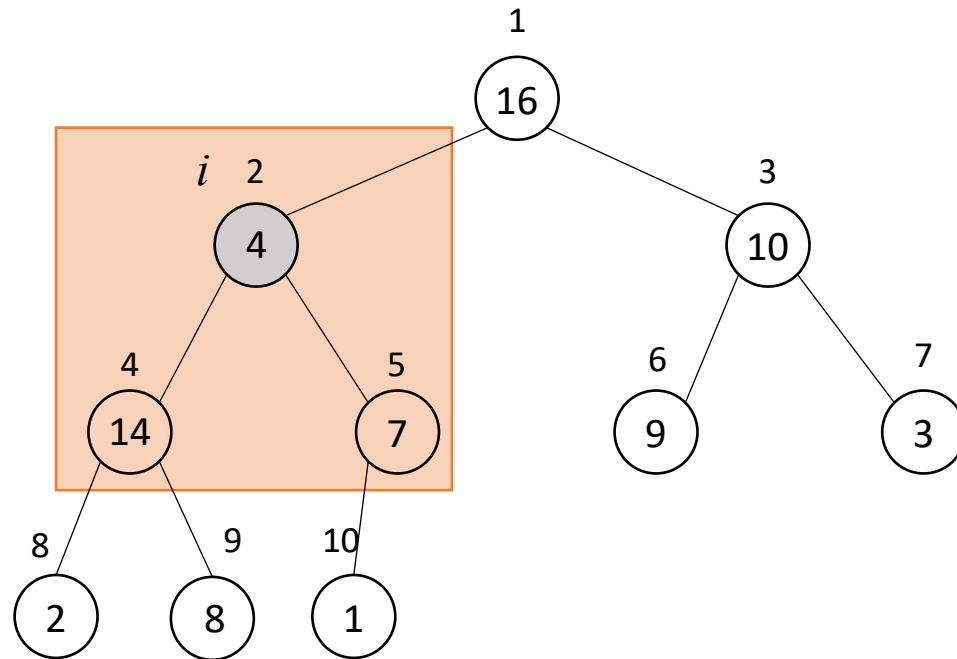
MAX-HEAPIFY(A, i)

```
1.  $l = \text{LEFT}(i)$ 
2.  $r = \text{RIGHT}(i)$ 
3. if  $l \leq A.\text{heap\_size}$  and  $A[l] > A[i]$ 
4.      $\text{largest} = l$ 
5. else  $\text{largest} = i$ 
6. if  $r \leq A.\text{heap\_size}$  and  $A[r] > A[\text{largest}]$ 
7.      $\text{largest} = r$ 
8. if  $\text{largest} \neq i$ 
9.     exchange  $A[i]$  with  $A[\text{largest}]$ 
10.    MAX-HEAPIFY( $A, \text{largest}$ )
```

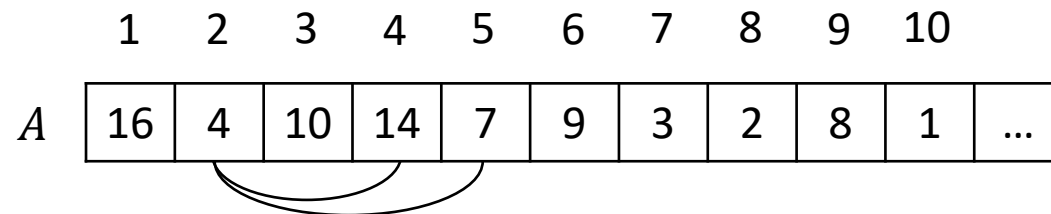
- Line 3 to 7: Determine the largest of elements $A[i]$, $A[\text{LEFT}(i)]$ and $A[\text{RIGHT}(i)]$; store its index in largest .
- If $A[i]$ is the largest, then the subtree rooted at node i is already a max-heap; terminates
- Otherwise, either one of the two children nodes has the largest elements, and $A[i]$ is swapped with $A[\text{largest}]$, which causes node i and its children to satisfy the max-heap property. (line 9)
- The node indexed by largest now has the original value $A[i]$, and thus the subtree rooted at largest might violate the max-heap property.
- Consequently, we call MAX-HEAPIFY recursively on that subtree. (line 10)

MAX-HEAPIFY example

MAX-HEAPIFY



$A.\text{heap-size} = 10$



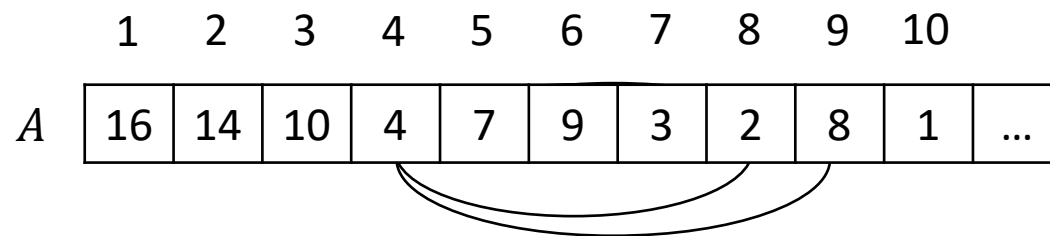
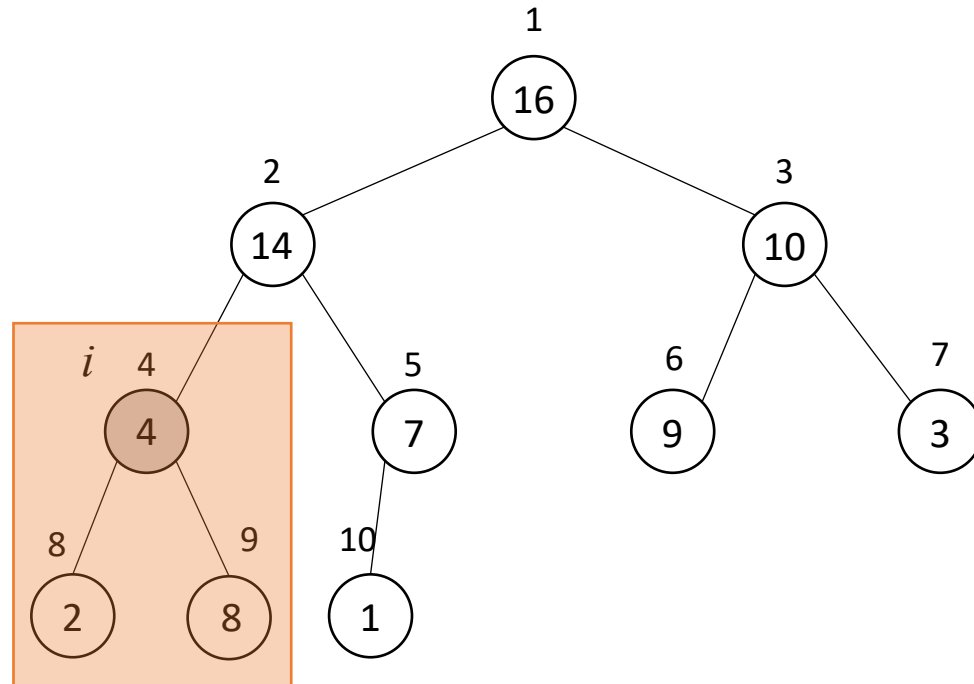
- Call MAX-HEAPIFY($A, i = 2$)
- $l = \text{LEFT}(2) = 4$
- $r = \text{RIGHT}(2) = 5$
 $\Rightarrow A[4] > A[5] > A[2]$
- $\text{largest} = 4, \rightarrow$ the subtree rooted at $A[2]$ violates the max-heap property
- Needs to exchange $A[4]$ with $A[2]$
- And then call MAX-HEAPIFY($A, 4$)

```

MAX-HEAPIFY( $A, i$ )
1.   $l = \text{LEFT}(i)$ 
2.   $r = \text{RIGHT}(i)$ 
3.  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4.     $\text{largest} = l$ 
5.  else  $\text{largest} = i$ 
6.  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7.     $\text{largest} = r$ 
8.  if  $\text{largest} \neq i$ 
9.    exchange  $A[i]$  with  $A[\text{largest}]$ 
10.   MAX-HEAPIFY( $A, \text{largest}$ )
    
```

MAX-HEAPIFY example

MAX-HEAPIFY

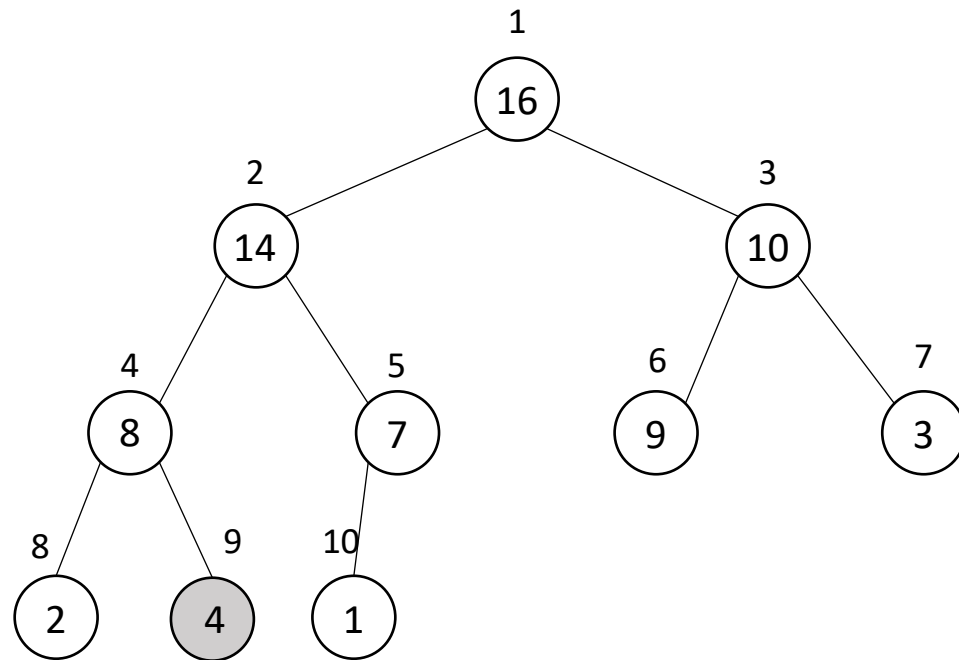


- Call MAX-HEAPIFY($A, i = 4$)
- $l = \text{LEFT}(4) = 8$
- $r = \text{RIGHT}(4) = 9$
- $A[9] > A[4] > A[8]$
- $largest = 9$, $\rightarrow$ the subtree rooted at $A[4]$ violates the max-heap property
- Needs to exchange $A[9]$ with $A[4]$
- And then call MAX-HEAPIFY($A, 9$)

```
MAX-HEAPIFY( $A, i$ )
1.   $l = \text{LEFT}(i)$ 
2.   $r = \text{RIGHT}(i)$ 
3.  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4.       $largest = l$ 
5.  else  $largest = i$ 
6.  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[largest]$ 
7.       $largest = r$ 
8.  if  $largest \neq i$ 
9.      exchange  $A[i]$  with  $A[largest]$ 
10.     MAX-HEAPIFY( $A, largest$ )
```

MAX-HEAPIFY example

MAX-HEAPIFY



- Call $\text{MAX-HEAPIFY}(A, i = 9)$
- $l = \text{LEFT}(9) = 18$
- $r = \text{RIGHT}(9) = 19$ } $> A.\text{heap-size} = 10$
- $\text{largest} = i = 9$, $\rightarrow$ the max-heap property is satisfied
- No further change needs to be made.

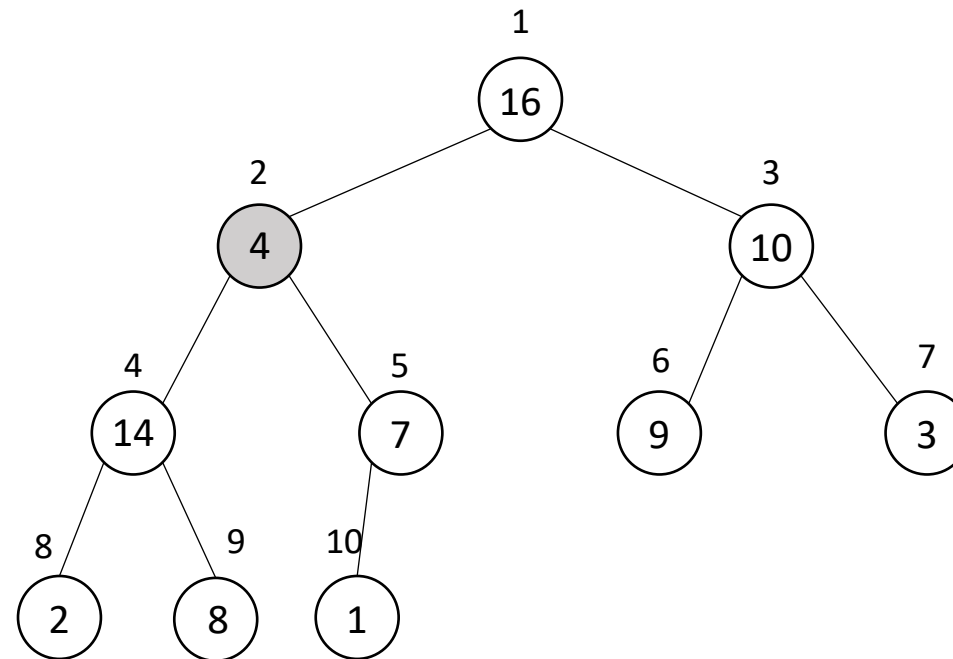
$A.\text{heap-size} = 10$

	1	2	3	4	5	6	7	8	9	10	
A	16	14	10	8	7	9	3	2	4	1	...

MAX-HEAPIFY example

MAX-HEAPIFY

- ❑ Initial call $\text{MAX-HEAPIFY}(A, i = 2)$ lets $A[2]$ “float down” to a proper position
- ❑ So that the subtree rooted at index $i = 2$ now obeys the max-heap property.



MAX-HEAPIFY running time

MAX-HEAPIFY

```
MAX-HEAPIFY(A, i)
1. l = LEFT(i)
2. r = RIGHT(i)
3. if l ≤ A.heap-size and A[l] > A[i]
4.     largest = l
5. else largest = i
6. if r ≤ A.heap-size and A[r] > A[largest]
7.     largest = r
8. if largest ≠ i
9.     exchange A[i] with A[largest]
10.    MAX-HEAPIFY(A, largest)
```

- MAX-HEAPIFY contains recursive call to itself.
- Its running time can be expressed with a recurrence.

All these steps (line 1 to 9) takes constant time, $\Theta(1)$

- Question: What's the size of the subproblem?
 - I.e., what's the size of the subtree rooted at node *largest*?

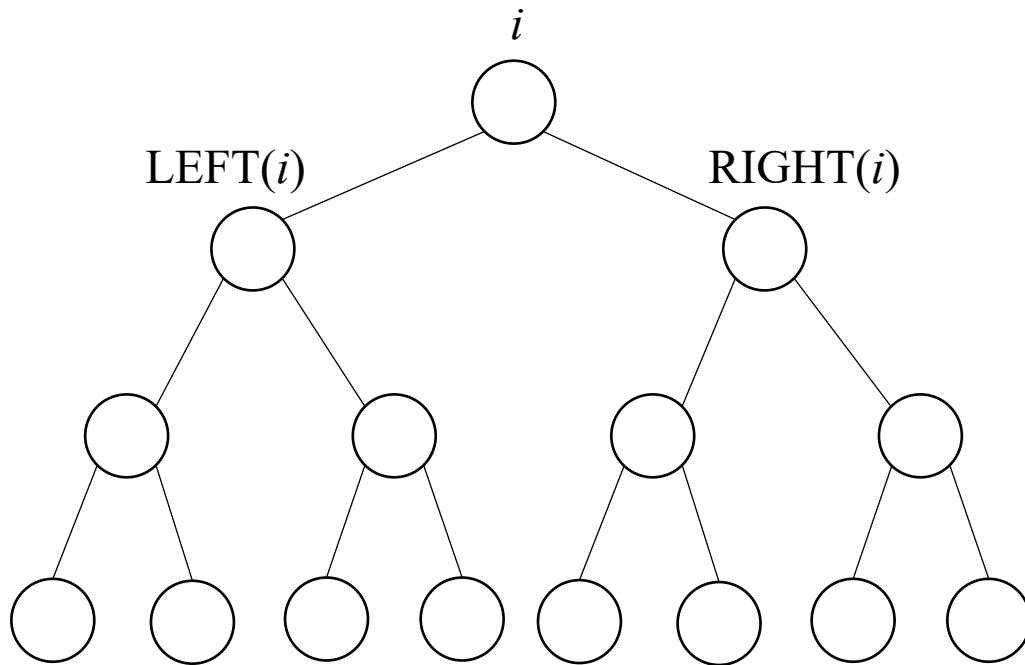
Running time: $T(n) = T\left(\frac{n}{b}\right) + \Theta(1)$

Size of subtree rooted at node *i*

Size of subtree rooted at node *largest*, *b* is unknown.

MAX-HEAPIFY running time (balanced case)

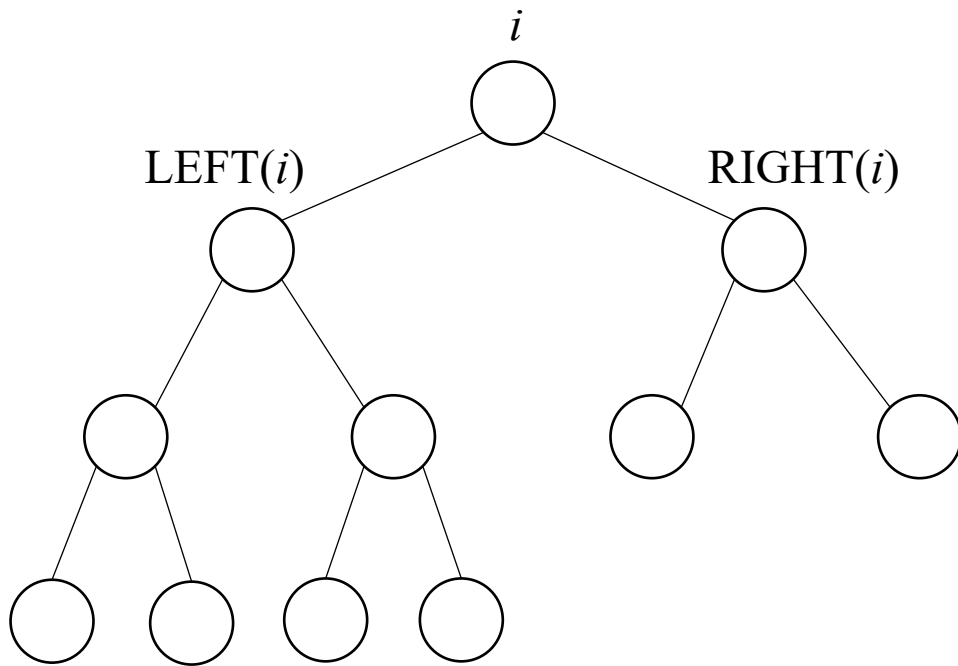
MAX-HEAPIFY



- The recursive call is **either** on the left subtree rooted at $\text{LEFT}(i)$, **or** the right subtree rooted at $\text{RIGHT}(i)$.
- In the most balanced case, bottom level is full
- Let n = size of root tree at node i .
- left subtree size = right subtree size = $\lfloor n/2 \rfloor$.
- $T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + \Theta(1) \leq T\left(\frac{n}{2}\right) + \Theta(1)$.
 - ➡ From master theorem, $T(n) = O(\lg n)$

MAX-HEAPIFY running time (unbalanced case)

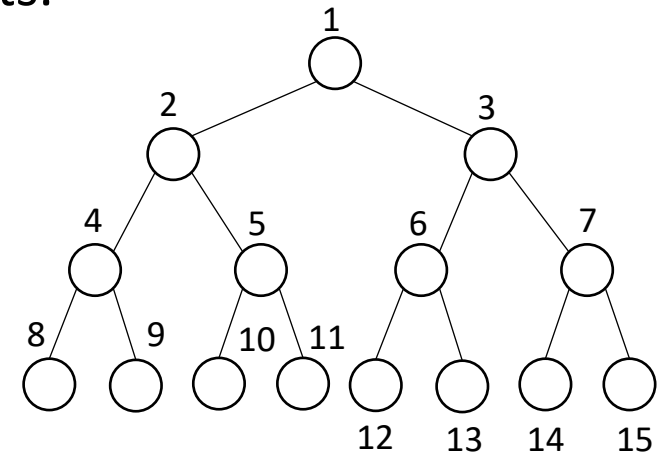
MAX-HEAPIFY



- Worst unbalanced case: bottom level is **half full**.
- Right subtree has one less level than the left subtree.
- Assume right subtree has k levels,
 - ✓ # of nodes in right subtree = $\sum_{i=0}^{k-1} 2^i = 2^k - 1 = R$
 - ✓ # of nodes in left subtree = $\sum_{i=0}^k 2^i = 2^{k+1} - 1 \approx 2R$
 - ✓ Total # of nodes $n \approx 3R$
- If largest = RIGHT(i), then $T(n) = T\left(\frac{n}{3}\right) + \Theta(1)$
- If largest = LEFT(i), then $T(n) = T\left(\frac{2n}{3}\right) + \Theta(1)$
- All together, $T(n) \leq T\left(\frac{2n}{3}\right) + \Theta(1)$
 - ➡ From master theorem, $T(n) = O(\lg n)$.

Building a max-heap

- ❑ Create a max-heap using MAX-HEAPIFY operation
 - Use the MAX-HEAPIFY in a bottom-up manner from an array $A[1:n]$. How?
 - A naïve solution: call MAX-HEAPIFY on every element of A , from $A[1]$ to $A[n]$.
 - This solution has **redundancy**.
- ❑ The elements in subarray $A[(\lfloor \frac{n}{2} \rfloor + 1) : n]$ are leaves of the tree.
 - No need to MAX-HEAPIFY them (because they are already 1-element heaps).
 - We just need to run MAX-HEAPIFY on non-leaf elements.

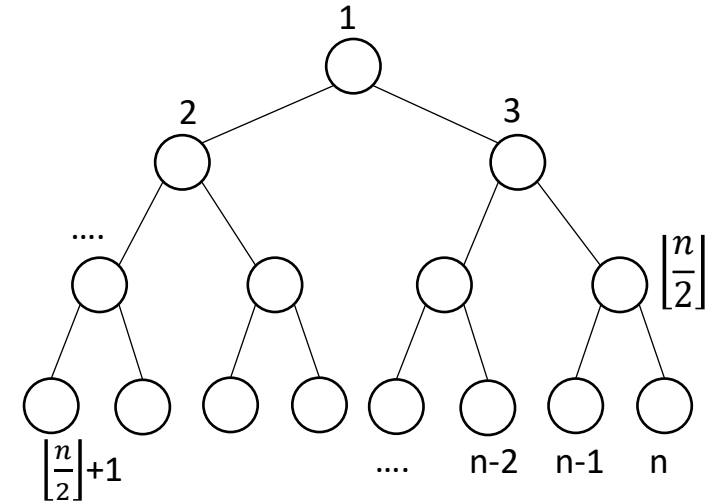


BUILD-MAX-HEAP procedure

❑ Pseudocode of BUILD-MAX-HEAP

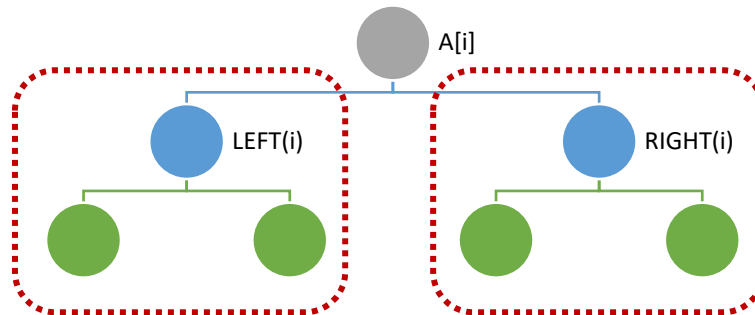
BUILD-MAX-HEAP(A)

1. $A.heap-size = A.length$
2. **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
3. MAX-HEAPIFY(A, i)



❑ Starting from the first non-leaf node $A[\lfloor n/2 \rfloor]$ down to the first node $A[1]$

- This **bottom-up** order is **important**, because MAX-HEAPIFY procedure can be applied to a node only if its *children* nodes are already max-heaps



BUILD-MAX-HEAP example

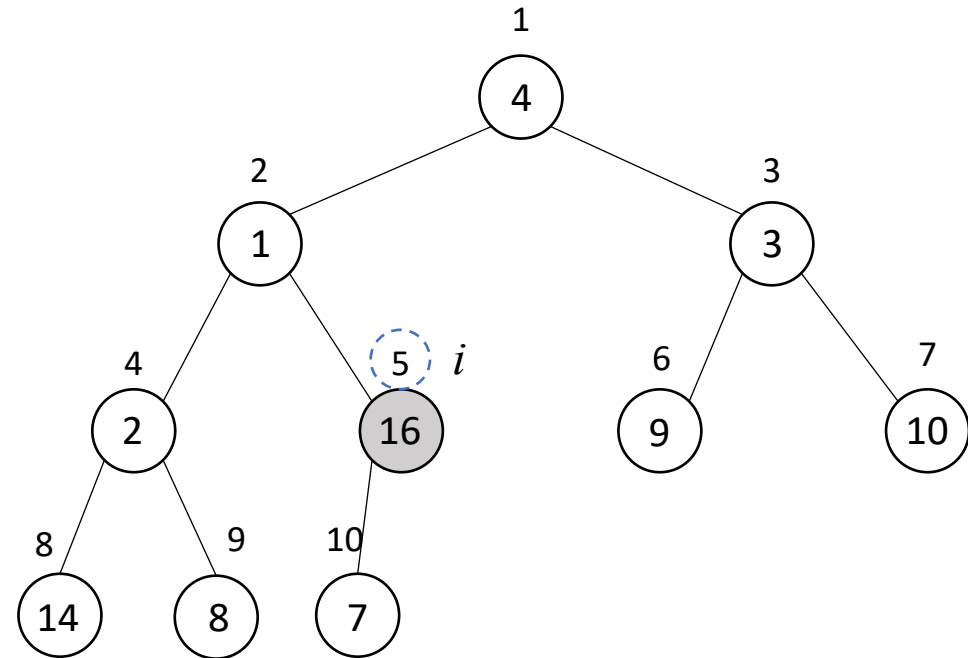
	1	2	3	4	5	6	7	8	9	10
<i>A</i>	4	1	3	2	16	9	10	14	8	7

$A.length = 10$

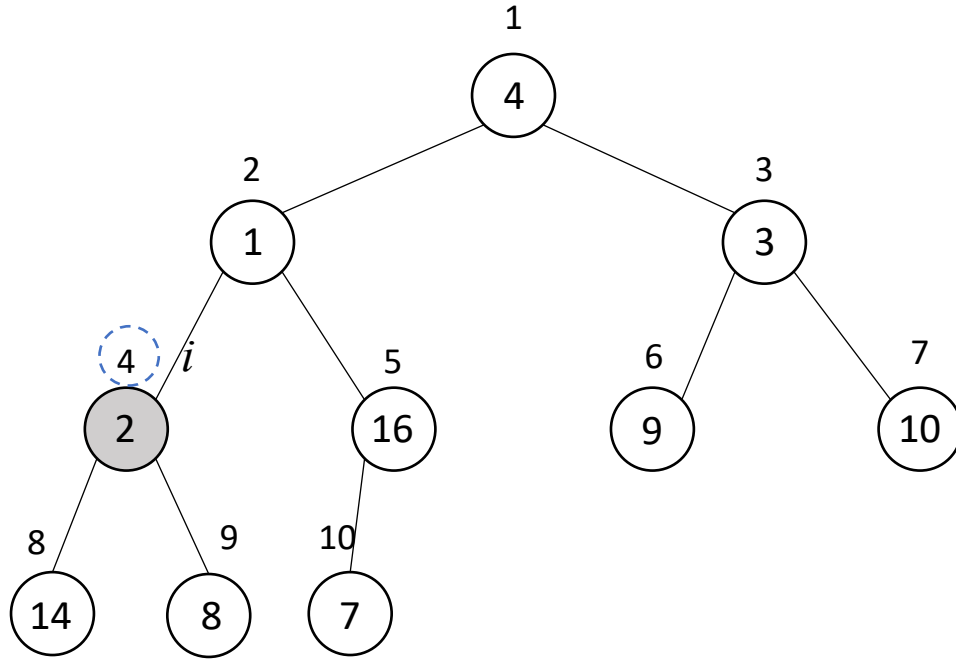
$\lfloor A.length/2 \rfloor = 5$

BUILD-MAX-HEAP(*A*)

1. $A.heap-size = A.length$
2. **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
3. MAX-HEAPIFY(*A*, *i*)



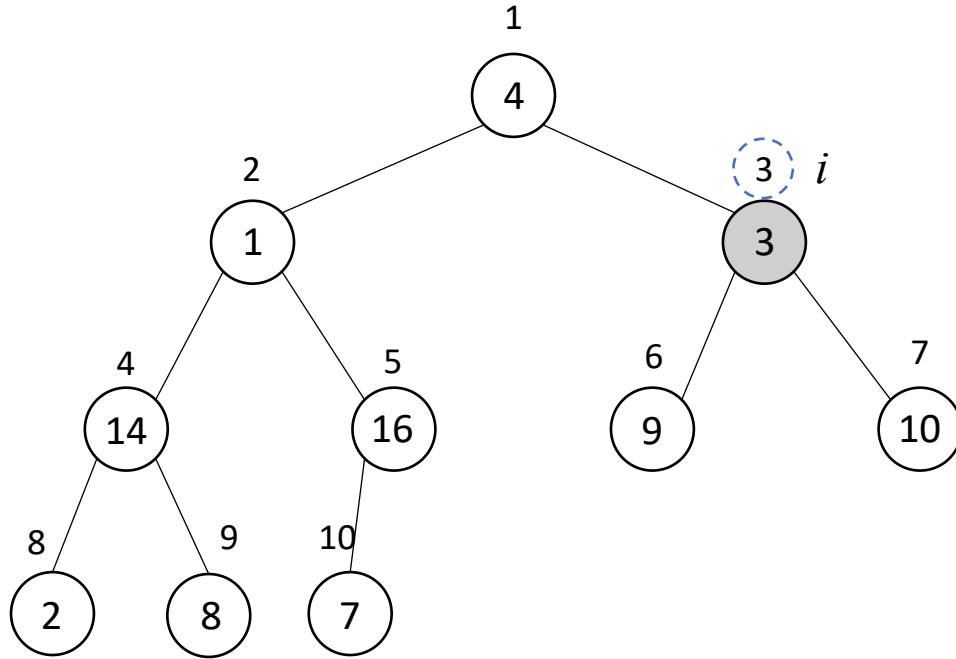
BUILD-MAX-HEAP example



BUILD-MAX-HEAP(A)

1. $A.heap\text{-}size = A.length$
2. **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
3. MAX-HEAPIFY(A, i)

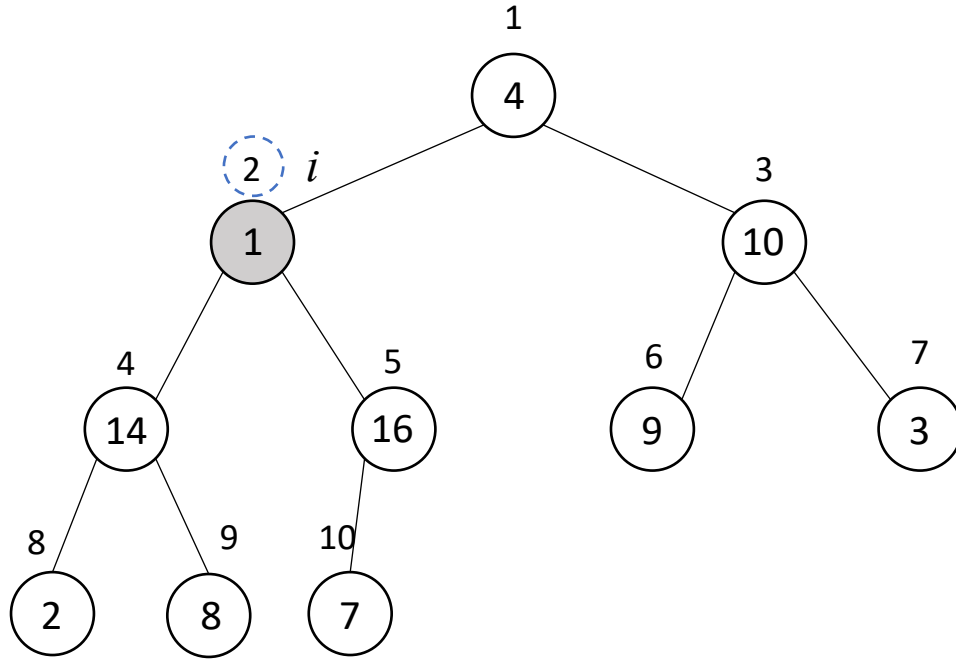
BUILD-MAX-HEAP example



BUILD-MAX-HEAP(A)

1. $A.heap\text{-}size = A.length$
2. **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
3. MAX-HEAPIFY(A, i)

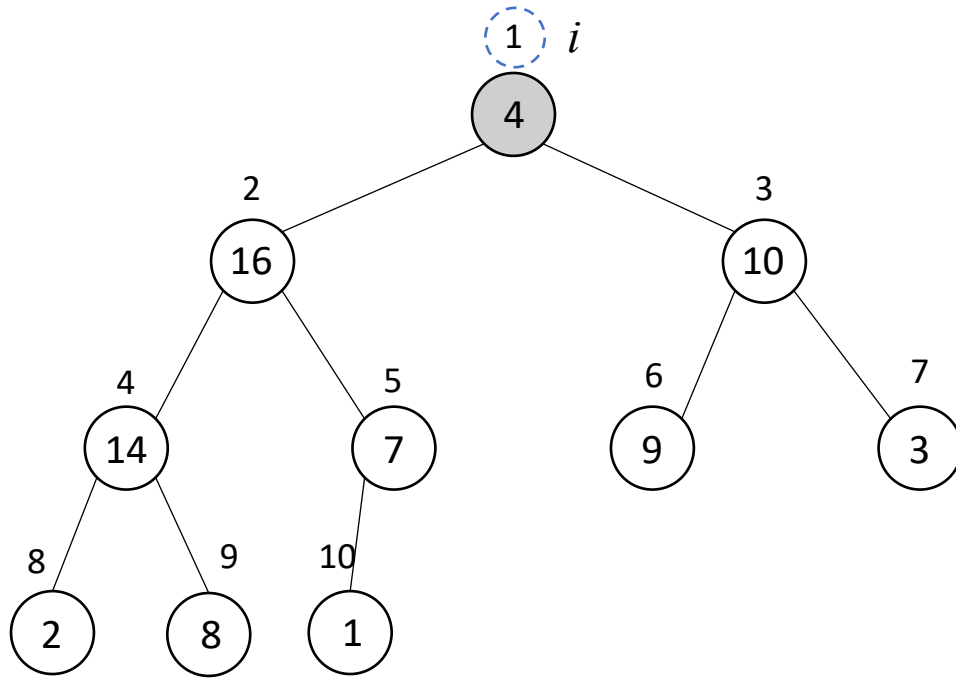
BUILD-MAX-HEAP example



BUILD-MAX-HEAP(A)

1. $A.heap\text{-}size = A.length$
2. **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
3. MAX-HEAPIFY(A, i)

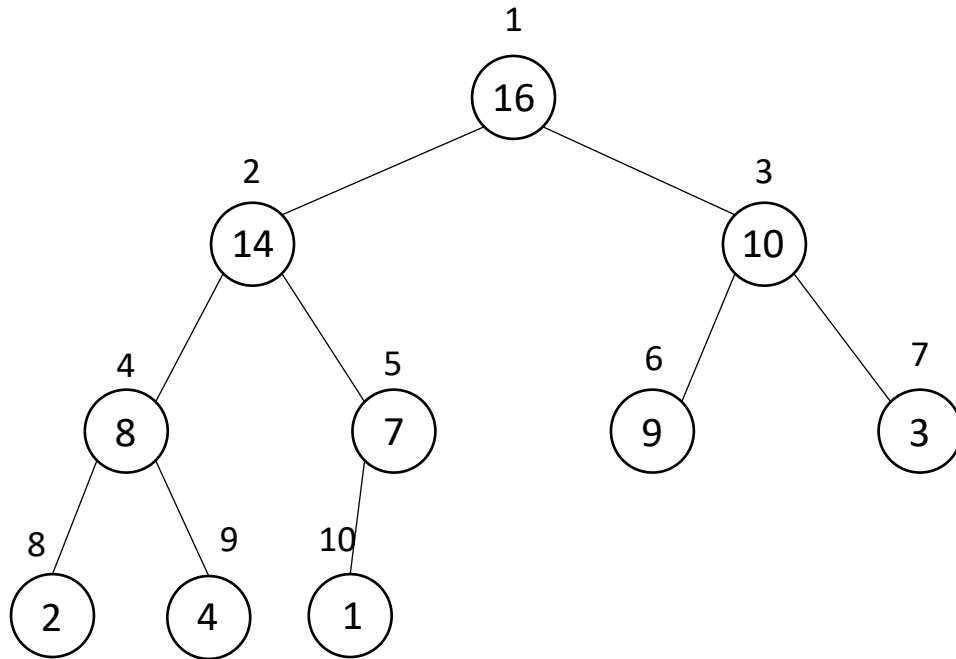
BUILD-MAX-HEAP example



BUILD-MAX-HEAP(A)

1. $A.heap\text{-}size = A.length$
2. **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
3. MAX-HEAPIFY(A, i)

BUILD-MAX-HEAP example



Now the whole array A is a max-heap

	1	2	3	4	5	6	7	8	9	10
A	16	14	10	8	7	9	3	2	4	1

BUILD-MAX-HEAP(A)

1. $A.heap\text{-}size = A.length$
2. **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
3. **MAX-HEAPIFY**(A, i)

Running time of BUILD-MAX-HEAP

BUILD-MAX-HEAP

- ❑ A simple upper bound

BUILD-MAX-HEAP(A)

1. $A.heap\text{-}size = A.length$
2. **for** $i = \lfloor A.length/2 \rfloor$ **downto** 1
3. MAX-HEAPIFY(A, i)

• Each call of MAX-HEAPIFY takes $O(\lg n)$ time.

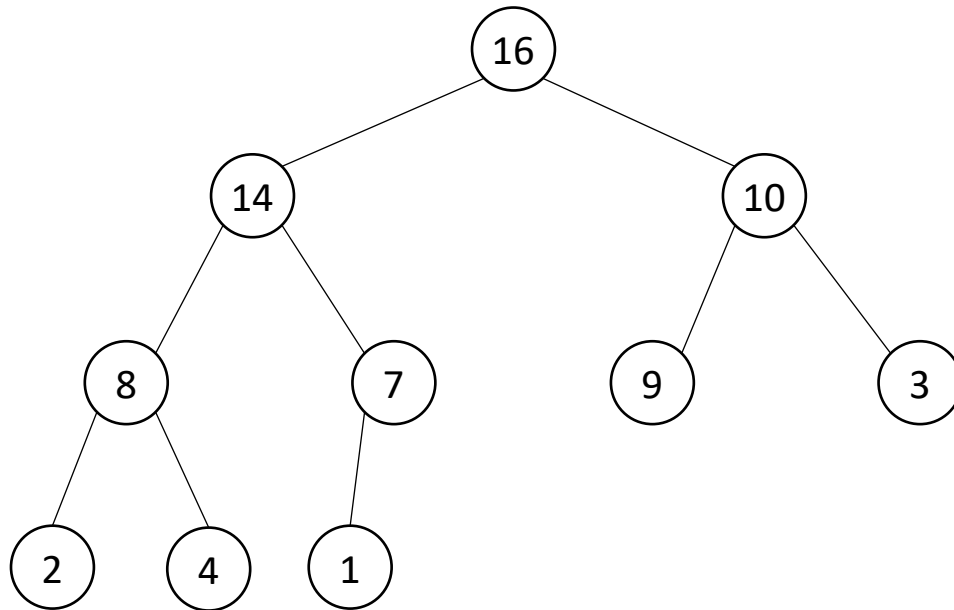
- ❑ BUILD-MAX-HEAP makes $\lfloor n/2 \rfloor$ such calls

➡ Thus, the running time is $O(n \lg n)$.

- ❑ This upper bound, though correct, is **not** asymptotically **tight**.

A tighter bound for BUILD-MAX-HEAP

BUILD-MAX-HEAP



- Observe that the running time of MAX-HEAPIFY(A, i) depends on the height of node i , i.e., $O(h)$.
- A heap of n elements has height $\lceil \lg n \rceil$.
- The number of nodes at height h is **at most** $\left\lceil \frac{n}{2^{h+1}} \right\rceil$.
- Then the total cost is bounded by:

$$\sum_{h=0}^{\lceil \lg n \rceil} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lceil \lg n \rceil} \frac{h}{2^h}\right)$$

A tighter bound for BUILD-MAX-HEAP

BUILD-MAX-HEAP

□ Because

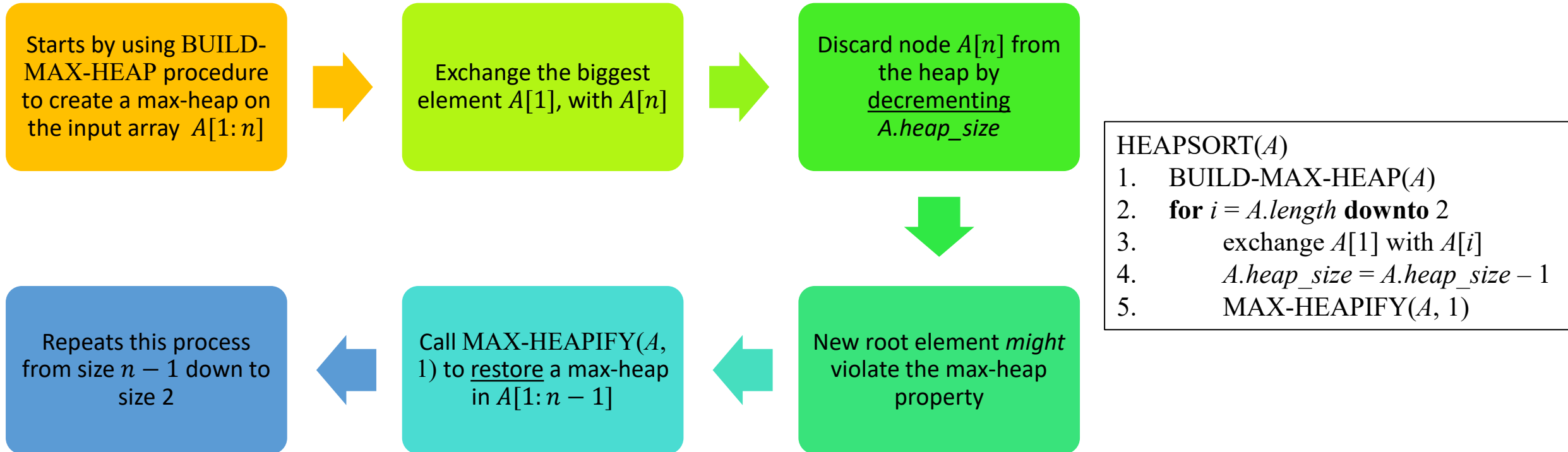
$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{\left(1 - \frac{1}{2}\right)^2} = 2 \quad \text{(taking a derivative of the sum of a geometric series)}$$
$$\sum_{k=0}^{\infty} k \cdot x^k = \frac{x}{(1-x)^2}, \text{ for } |x| < 1$$

□ Then

$$O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(n)$$

□ Therefore, we can build a max-heap from an unordered array in linear time. 😊

Heapsort algorithm

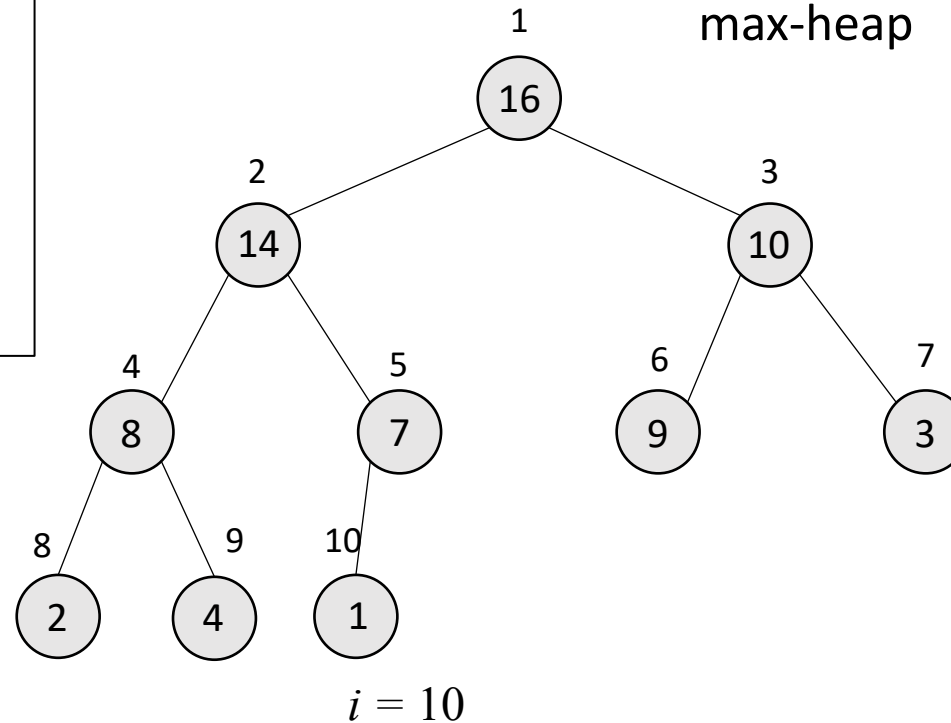


HEAPSORT example

HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. **for** $i = A.length$ **downto** 2
3. exchange $A[1]$ with $A[i]$
4. $A.heap_size = A.heap_size - 1$
5. MAX-HEAPIFY($A, 1$)

- After line 1 building the max-heap

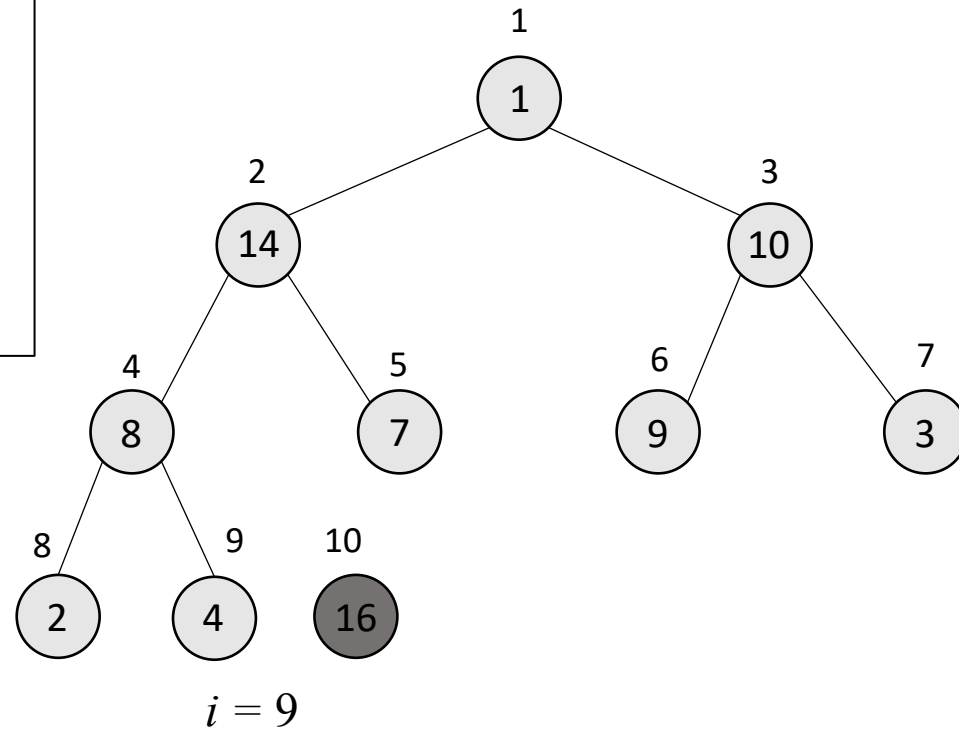


HEAPSORT example

After line 3 to 4

HEAPSORT(A)

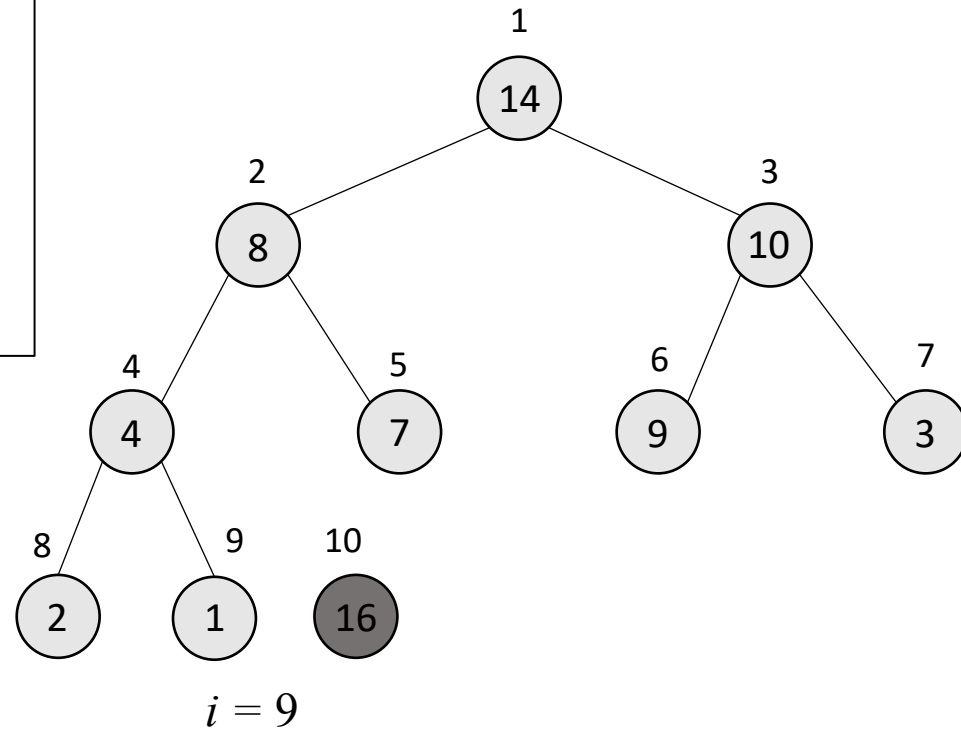
1. BUILD-MAX-HEAP(A)
2. **for** $i = A.length$ **downto** 2
3. exchange $A[1]$ with $A[i]$
4. $A.heap-size = A.heap-size - 1$
5. MAX-HEAPIFY($A, 1$)



HEAPSORT example

HEAPSORT(A)

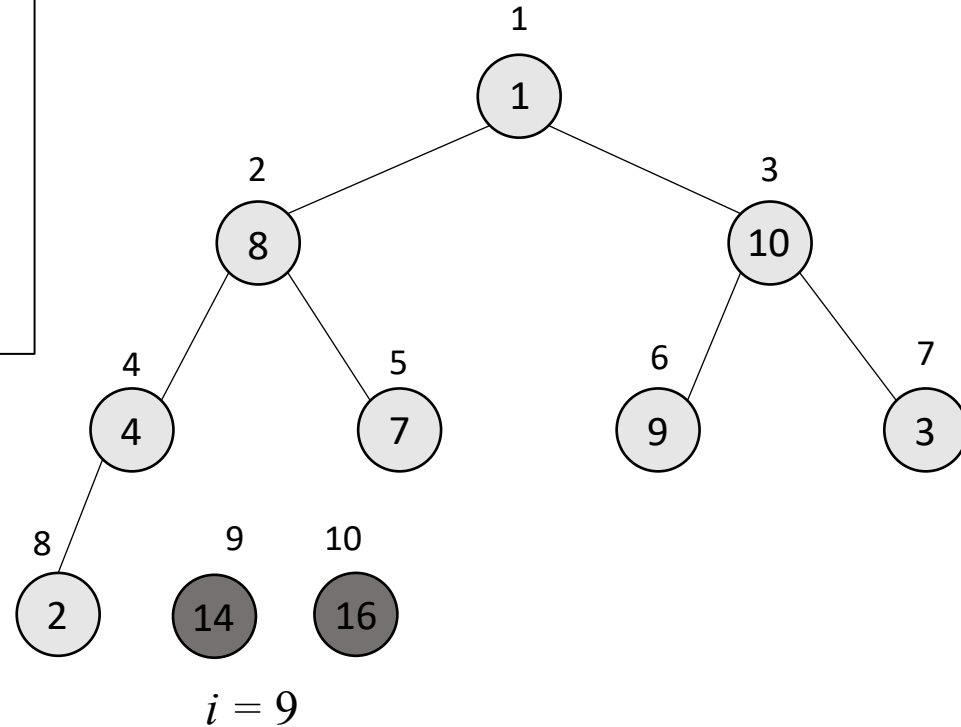
1. BUILD-MAX-HEAP(A)
2. **for** $i = A.length$ **downto** 2
3. exchange $A[1]$ with $A[i]$
4. $A.heap\text{-}size = A.heap\text{-}size - 1$
5. MAX-HEAPIFY($A, 1$)



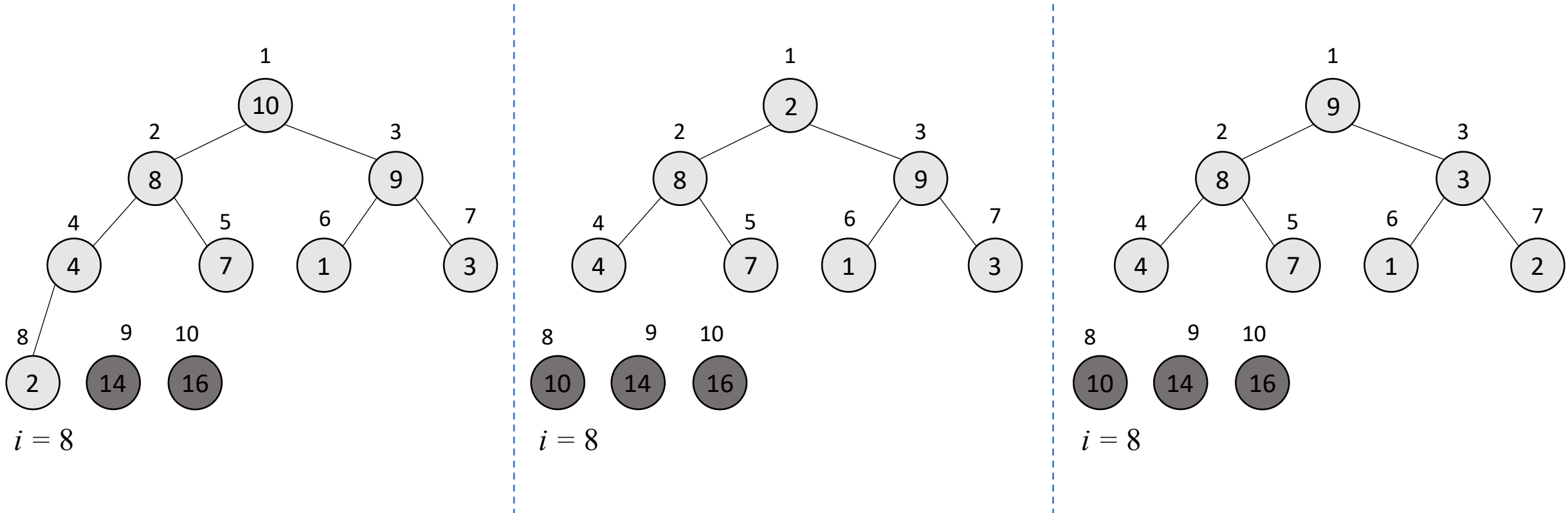
HEAPSORT example

HEAPSORT(A)

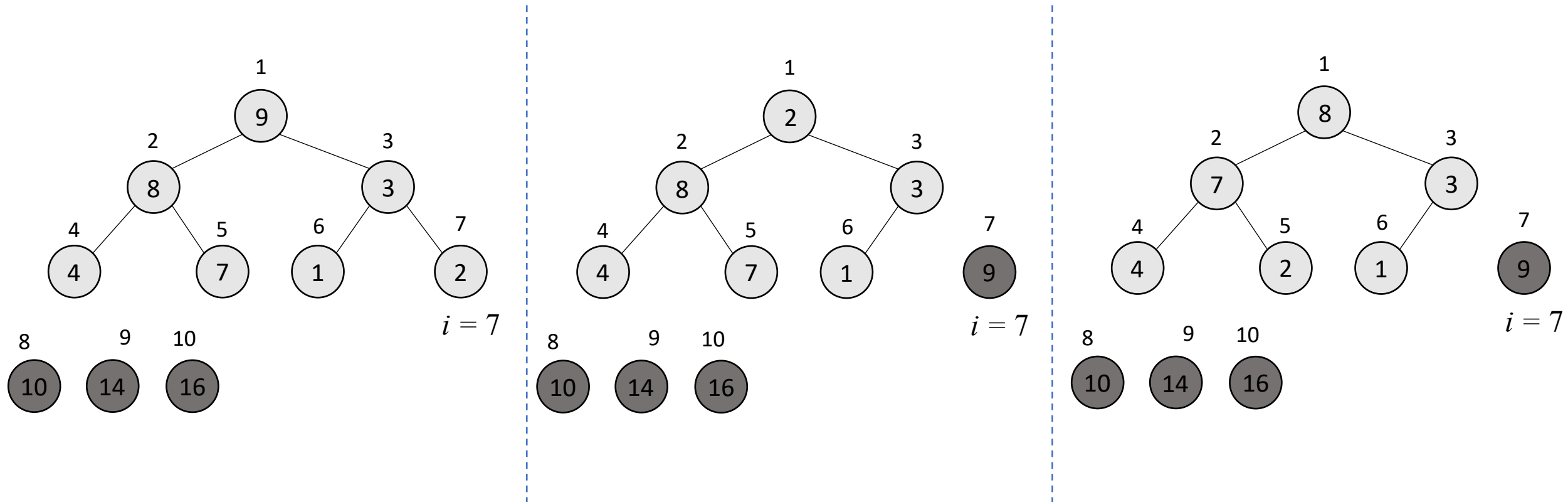
1. BUILD-MAX-HEAP(A)
2. **for** $i = A.length$ **downto** 2
3. exchange $A[1]$ with $A[i]$
4. $A.heap\text{-}size = A.heap\text{-}size - 1$
5. MAX-HEAPIFY($A, 1$)



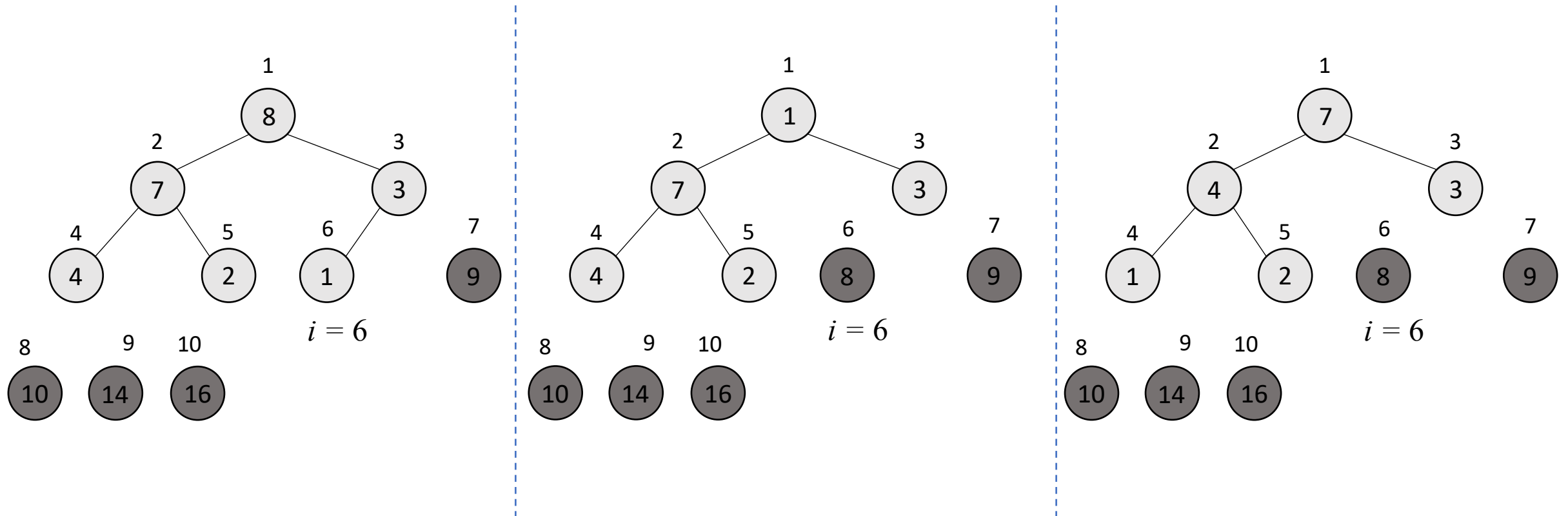
HEAPSORT example



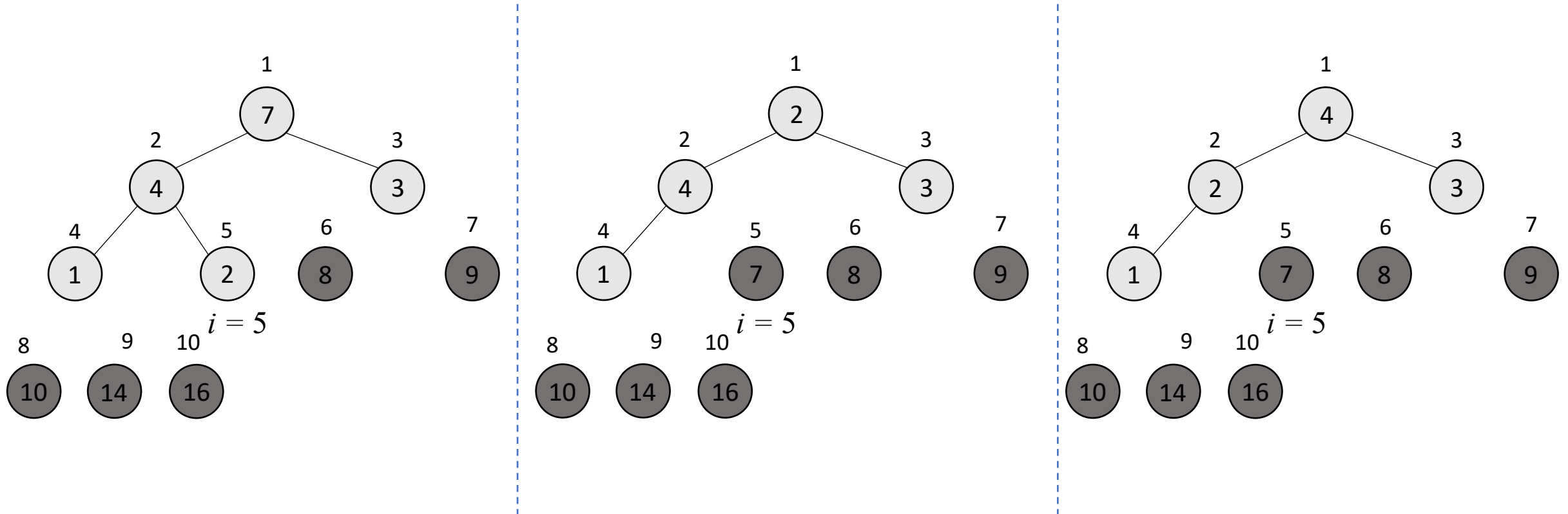
HEAPSORT example



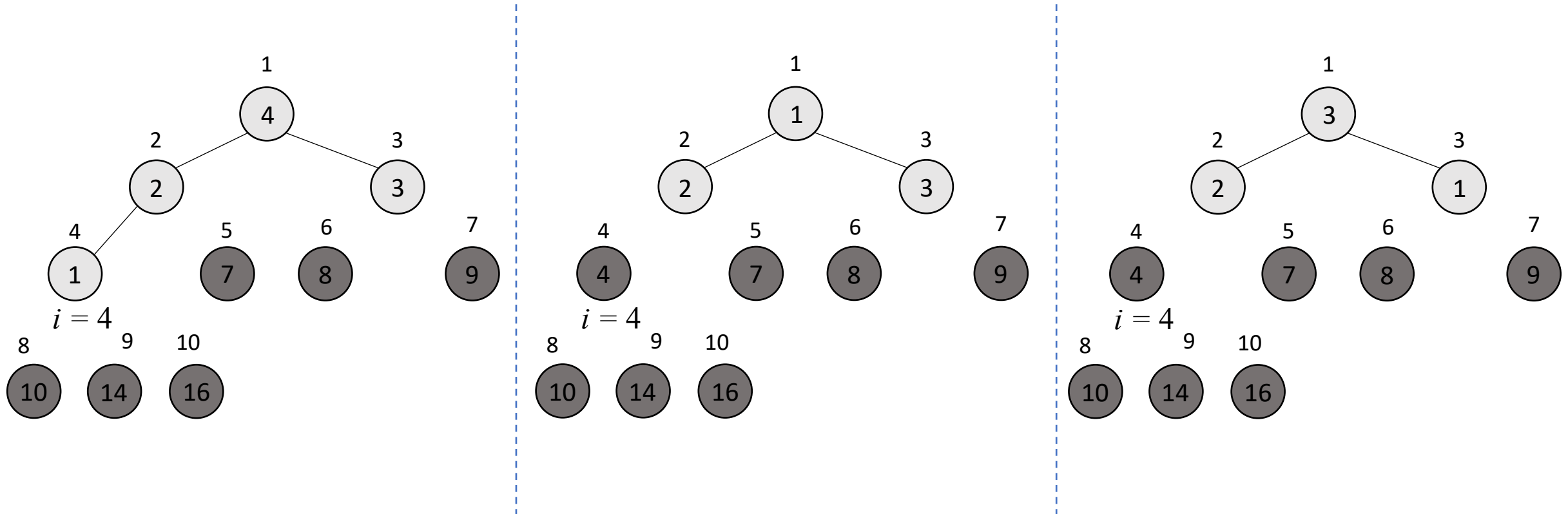
HEAPSORT example



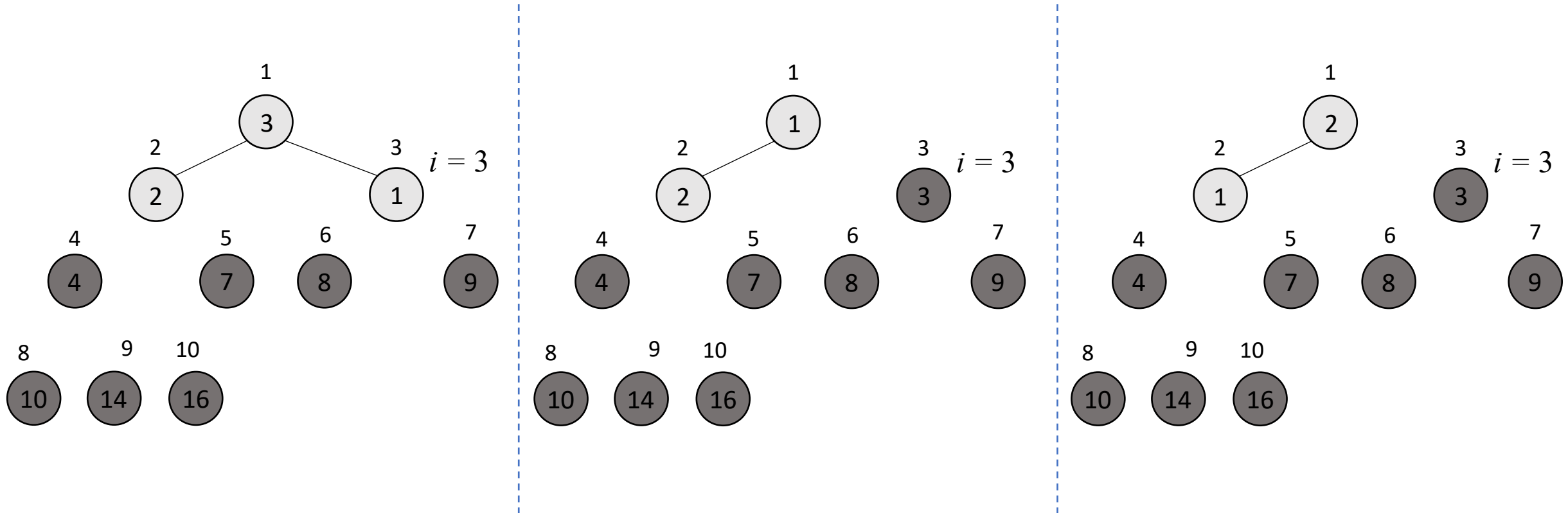
HEAPSORT example



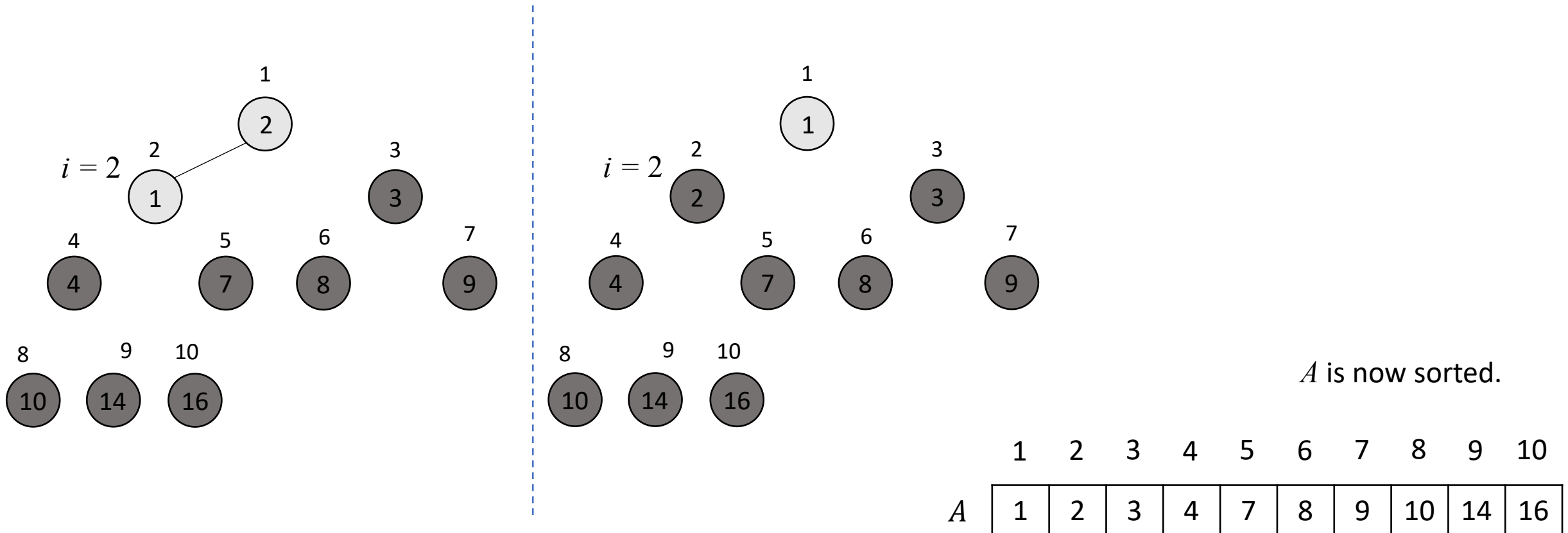
HEAPSORT example



HEAPSORT example



HEAPSORT example



Running time of HEAPSORT

HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. **for** $i = A.length$ **downto** 2
3. exchange $A[1]$ with $A[i]$
4. $A.heap-size = A.heap-size - 1$
5. MAX-HEAPIFY($A, 1$)

- BUILD-MAX-HEAP takes $O(n)$ time

- Each of the $n - 1$ calls to MAX-HEAPIFY takes $O(\lg n)$ time

- Therefore, the total running time is $O(n \lg n)$

Example

Problem

❑ Consider the following array of elements

[70, 23, 60, 19, 13, 16, 1, 4, 8, 12, 7, 10, 85]

What is the minimum number of interchanges (swap) of nodes that are needed to convert into a max-heap?

